



**The University of Azad Jammu and Kashmir,
Muzaffarabad**

Name: Muneeba Arshad

Roll NO: 2024-SE-06

Semester:4th

Course Title: Database System

Course Code: CS-2204

Submitted To: Engr. Awais Rathore

Department of Software Engineering

Table of Contents

1. Assignment Requirements	3
2. Task 1 — Database Design and Implementation.....	3
2.1 Constraints	4
2.2 Efficiency Improvements in the Design	4
3. Task 2 — Normalization	5
3.1 1NF	5
3.2 2NF	5
3.3 3NF	5
4. Task 3 — JOIN Queries	5
Q1 — Every Rental	6
Q2 — All Customers	6
Q3 — All Vehicles	7
Q4 — Rental Count.....	8
5. Task 4 — VIEW.....	8
6. Task 5 — TRIGGER.....	9
7. Task 6 — STORED PROCEDURE.....	10
8. Task 7 — Optimization Analysis	11
9. Testing and Screenshot Checklist.....	12
10. Marking Rubric Alignment.....	13
11. Submission Files	13
12. Conclusion	13

DATABASE MANAGEMENT SYSTEM OPEN-ENDED LAB REPORT

CAR RENTAL MANAGEMENT SYSTEM

1. Assignment Requirements

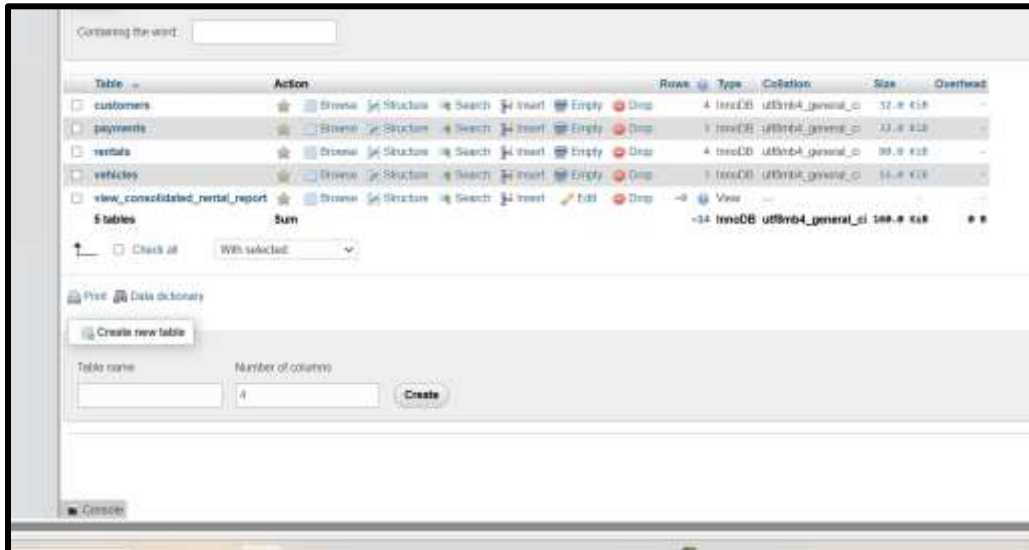
The supplied manual requires a complete relational database solution for CarGo Rentals. The company currently stores customer, vehicle, rental, return, and payment information in spreadsheets, which has created duplicate information, inconsistent records, and reporting difficulties. The database must manage customers and vehicles, record rentals and returns, calculate rental charges, maintain payments, prevent a rented vehicle from being rented again, and support rental-history/current-rental/vehicle-activity reporting. □filecite□turn0file0□L14-L23□

The manual's required work is: database design and implementation; normalization from 1NF to 3NF; four JOIN queries; one VIEW; a TRIGGER; a STORED PROCEDURE; and optimization analysis. The submission must contain an SQL file, a short report, and screenshots/output demonstrating the required database features.

2. Task 1 — Database Design and Implementation

The database is decomposed into four base tables: Customers, Vehicles, Rentals, and Payments. This keeps each type of business fact in its appropriate relation and avoids storing customer and vehicle details repeatedly in every rental record.

Table	Primary Key	Foreign Key(s)	Main Purpose
Customers	CustomerID	None	Customer identity and contact information.
Vehicles	VehicleID	None	Vehicle identity, model, rate, and availability.
Rentals	RentalID	CustomerID, VehicleID	Rental transaction, dates, days, and charge.
Payments	PaymentID	RentalID	Payment amount and payment status.



2.1 Constraints

- PRIMARY KEY: uniquely identifies every row.
- FOREIGN KEY: prevents orphan rental/payment records.
- NOT NULL: ensures required fields are supplied.
- UNIQUE: prevents duplicate phone numbers, emails, licenses, and vehicle numbers.
- CHECK: prevents invalid rates, negative payments/charges, and invalid rental dates.
- DEFAULT: supplies sensible default values such as Available vehicle status and payment date/status.
- ENUM: restricts status and payment-method values to defined business options.

2.2 Efficiency Improvements in the Design

- INT UNSIGNED keys reduce unnecessary key-space overhead for positive identifiers.
- Foreign-key columns are indexed because they are frequently used in JOIN operations.
- A composite index on (VehicleID, ReturnDate) supports the current-rental lookup used by Task 3.
- A composite index on (CustomerID, RentalDate) supports customer rental-history queries.
- RentalDays is stored when the rental is registered, avoiding repeated DATEDIFF calculations in reports.
- The stored procedure uses a transaction and SELECT ... FOR UPDATE so the selected vehicle is locked while its rental is created.
- The design avoids redundant customer and vehicle attributes inside Rentals, reducing update anomalies and storage duplication.

3. Task 2 — Normalization

The manual gives the original single-record structure as: RentalID, CustomerName, CustomerPhone, VehicleNumber, VehicleModel, DailyRate, RentalDate, ReturnDate, and PaymentAmount. It specifically asks for conversion to 1NF, 2NF, and 3NF and an explanation of the redundancy, dependencies, and anomalies removed at each stage.

3.1 1NF

1NF requires atomic values and no repeating groups. The supplied sample already contains single values in each field, so the main improvement is to treat each rental as one distinct row identified by RentalID. Repeating customer or vehicle information is still present across multiple rentals, so redundancy remains.

3.2 2NF

For this design, RentalID identifies a rental. Customer details and vehicle details should not be copied into every rental row because they are facts about customers and vehicles. They are separated into Customers and Vehicles, while Rentals stores CustomerID and VehicleID as foreign keys. This reduces repeated information and update anomalies.

3.3 3NF

3NF removes dependencies in which a non-key attribute depends on another non-key attribute. CustomerName, CustomerPhone, and DriverLicenseNo depend on CustomerID; VehicleModel and DailyRate depend on VehicleID; rental dates and charge depend on RentalID; payment information is separated into Payments. The resulting relations therefore have clearer dependencies and reduced insertion, update, and deletion anomalies.

Stage	Structure	Main Problem	Improvement
1NF	Atomic rental rows	Customer/vehicle data repeats	Ensure atomic values and unique rental records
2NF	Customers + Vehicles + Rentals	Repeated entity data	Move entity facts to their own tables
3NF	Customers + Vehicles + Rentals + Payments	Non-key dependency/redundancy	Separate payment facts and preserve key-based dependencies

4. Task 3 — JOIN Queries

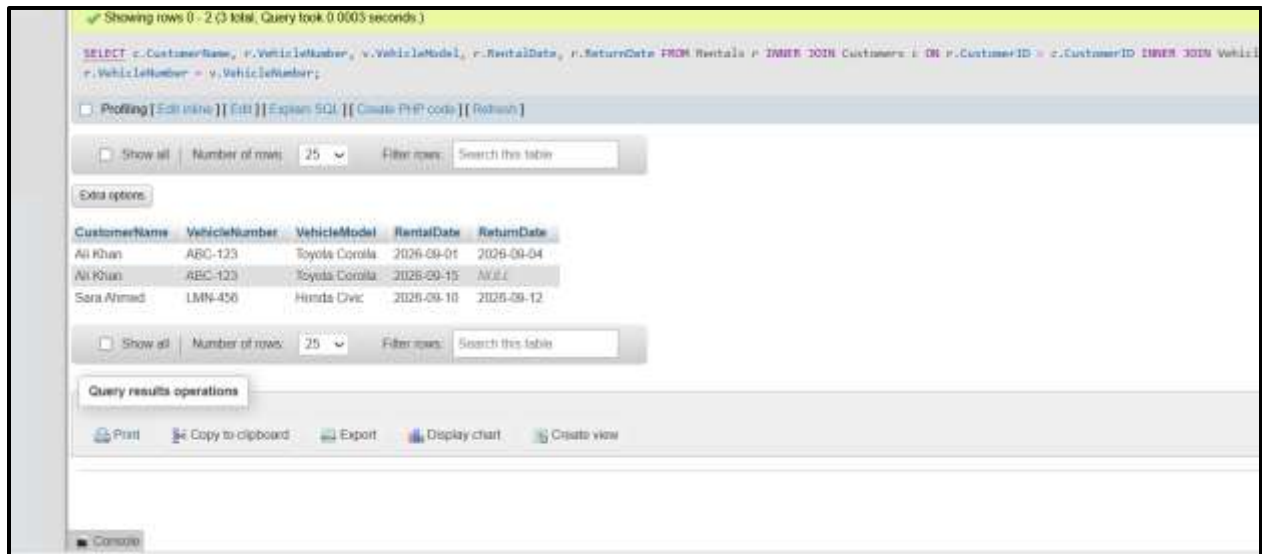
The manual requires four business queries and requires each to have its SQL statement plus screenshot/output of the result.

Car_Rental_Management_System

Q1 — Every Rental

INNER JOIN connects Rentals to Customers and Vehicles and displays customer name, vehicle number/model, rental date, and return date.

```
SELECT c.CustomerName, r.VehicleNumber, v.VehicleModel, r.RentalDate, r.ReturnDate
FROM Rentals r
INNER JOIN Customers c ON r.CustomerID = c.CustomerID
INNER JOIN Vehicles v ON r.VehicleNumber = v.VehicleNumber;
```



Showing rows 0 - 2 (3 total, Query took 0.0003 seconds.)

```
SELECT c.CustomerName, r.VehicleNumber, v.VehicleModel, r.RentalDate, r.ReturnDate FROM Rentals r INNER JOIN Customers c ON r.CustomerID = c.CustomerID INNER JOIN Vehicles v ON r.VehicleNumber = v.VehicleNumber;
```

☐ Profiling ☐ Edit table ☐ Edit ☐ Explain SQL ☐ Create PHP code ☐ Refresh

☐ Show all Number of rows: 25 Filter rows: Search this table

Extra options:

CustomerName	VehicleNumber	VehicleModel	RentalDate	ReturnDate
Ali Khan	ABC-123	Toyota Corolla	2026-09-01	2026-09-04
Ali Khan	ABC-123	Toyota Corolla	2026-09-15	NULL
Sara Ahmed	LMN-456	Honda Civic	2026-09-10	2026-09-12

☐ Show all Number of rows: 25 Filter rows: Search this table

Query results operations

Console

Q2 — All Customers

LEFT JOIN starts from Customers so customers with no rental are not lost.

```
SELECT c.CustomerName, r.RentalID, r.VehicleNumber
FROM Customers c
LEFT JOIN Rentals r ON c.CustomerID = r.CustomerID;
```

Car_Rental_Management_System

The screenshot shows a database query interface with the following SQL query:

```
SELECT c.CustomerName, r.RentalID, r.VehicleNumber FROM Customers c LEFT JOIN Rentals r ON c.CustomerID = r.CustomerID;
```

Below the query, there are controls for "Show all", "Number of rows" (set to 25), and a "Filter rows" search bar. The query results are displayed in a table:

CustomerName	RentalID	VehicleNumber
Ali Khan	R001	ABC-123
Sara Ahmed	R002	LMN-456
Ali Khan	R003	ABC-123
Omer Farooq	NULL	NULL
Zanab Bibi	NULL	NULL

Below the table, there are more controls for "Show all", "Number of rows" (set to 25), and a "Filter rows" search bar. At the bottom, there are "Query results operations" including Print, Copy to clipboard, Export, Display chart, and Create view. The interface also includes a "Console" tab and a system tray at the bottom showing various application icons and weather information (24°C, Partly cloudy).

Q3 — All Vehicles

LEFT JOIN starts from Vehicles and places the current-rental condition in the JOIN so vehicles with no active rental remain visible.

```
SELECT v.VehicleNumber, v.VehicleModel, v.Status, r.RentalID, r.RentalDate
FROM Vehicles v
LEFT JOIN Rentals r ON v.VehicleNumber = r.VehicleNumber AND r.ReturnDate IS NULL;
```

The screenshot shows a database query interface with the following SQL query:

```
SELECT v.VehicleNumber, v.VehicleModel, v.Status, r.RentalID, r.RentalDate FROM Vehicles v LEFT JOIN Rentals r ON v.VehicleNumber = r.VehicleNumber AND r.ReturnDate IS NULL;
```

Below the query, there are controls for "Show all", "Number of rows" (set to 25), and a "Filter rows" search bar. The query results are displayed in a table:

VehicleNumber	VehicleModel	Status	RentalID	RentalDate
ABC-123	Toyota Corolla	Rented	R003	2025-09-15
LMN-456	Honda Civic	Available	NULL	NULL
XYZ-789	Suzuki Alto	Available	NULL	NULL

Below the table, there are more controls for "Show all", "Number of rows" (set to 25), and a "Filter rows" search bar. At the bottom, there are "Query results operations" including Print, Copy to clipboard, Export, Display chart, and Create view. The interface also includes a "Console" tab and a system tray at the bottom showing various application icons and weather information (24°C).

Car_Rental_Management_System

Q4 — Rental Count

LEFT JOIN plus COUNT(RentalID) returns zero for customers who have never rented.

```
SELECT c.CustomerName, COUNT(r.RentalID) AS TotalRentals
FROM Customers c
LEFT JOIN Rentals r ON c.CustomerID = r.CustomerID
GROUP BY c.CustomerID, c.CustomerName;
```



CustomerName	TotalRentals
Ali Khan	2
Sara Ahmad	1
Omar Farooq	0
Zahrah Hidi	3

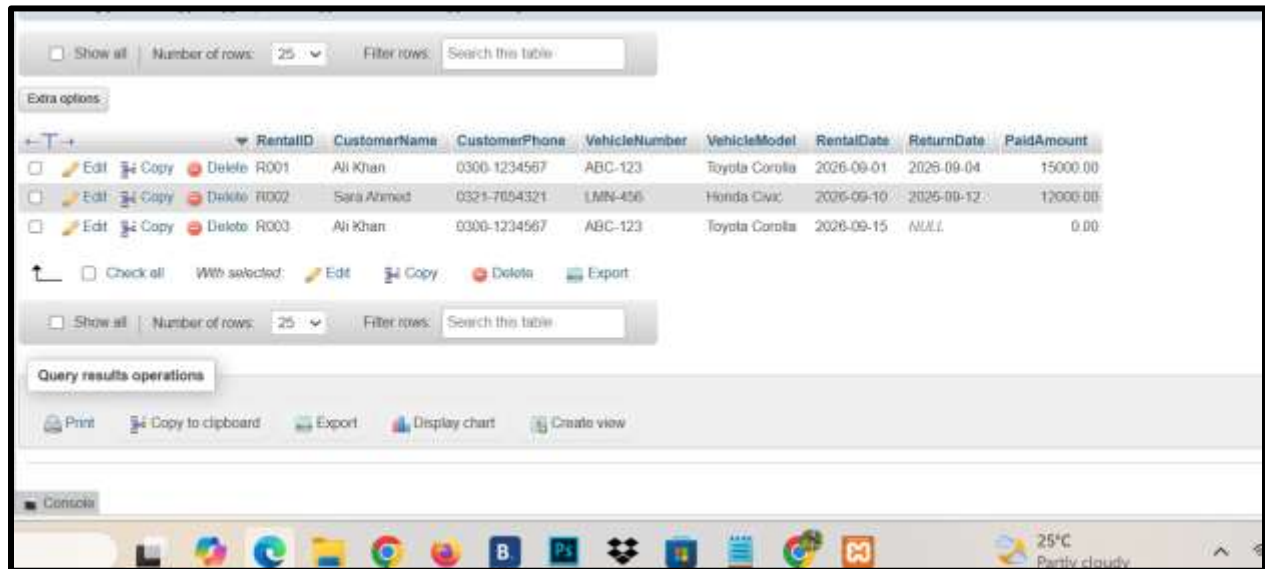
5. Task 4 — VIEW

RentalReport is a consolidated view containing customer, vehicle, rental, and payment information. It is useful because management can query one reusable object instead of rewriting the same multi-table JOIN each time a rental report is needed. The manual explicitly asks for one useful VIEW and a 2–3 sentence explanation.

```
CREATE VIEW View_Consolidated_Rental_Report AS
SELECT r.RentalID, c.CustomerName, c.CustomerPhone, v.VehicleNumber, v.VehicleModel, r.RentalDate, r.ReturnDate,
COALESCE(p.PaymentAmount, 0.00) AS PaidAmount
FROM Rentals r
INNER JOIN Customers c ON r.CustomerID = c.CustomerID
INNER JOIN Vehicles v ON r.VehicleNumber = v.VehicleNumber
LEFT JOIN Payments p ON r.RentalID = p.RentalID;
```

Car_Rental_Management_System

Testing View Command: SELECT * FROM View_Consolidated_Rental_Report;



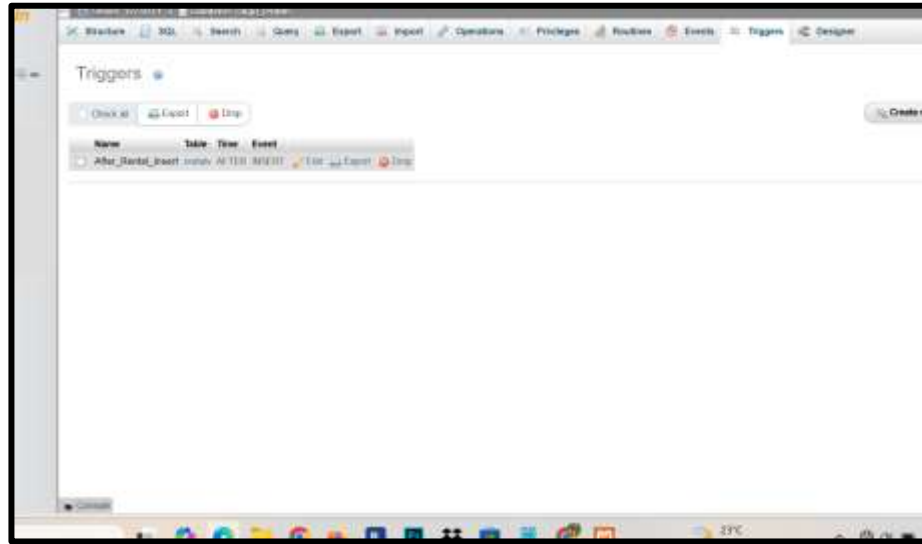
	RentalID	CustomerName	CustomerPhone	VehicleNumber	VehicleModel	RentalDate	ReturnDate	PaidAmount
☐	R001	Ali Khan	0300-1234567	ABC-123	Toyota Corolla	2026-09-01	2026-09-04	15000.00
☐	R002	Sara Ahmed	0321-7654321	LMN-456	Honda Civic	2026-09-10	2026-09-12	12000.00
☐	R003	Ali Khan	0300-1234567	ABC-123	Toyota Corolla	2026-09-15	NULL	0.00

6. Task 5 — TRIGGER

The supplied manual labels the sections as Task 4 (VIEW) and Task 6 (Stored Procedure), but the submission requirements explicitly require a TRIGGER as well.

Two triggers are implemented. The BEFORE INSERT trigger rejects a rental when its vehicle is not Available. The AFTER UPDATE trigger changes a vehicle back to Available when a return date is recorded. This directly supports the scenario rule that a vehicle should not be available for another rental while it is already rented.

```
CREATE TRIGGER After_Rental_Insert
AFTER INSERT ON Rentals
FOR EACH ROW
BEGIN
    UPDATE Vehicles
    SET Status = 'Rented'
    WHERE VehicleNumber = NEW.VehicleNumber;
```



END;

7. Task 6 — STORED PROCEDURE

RegisterRental accepts a customer, vehicle, rental date, and return date. It validates the dates, verifies the customer and vehicle, locks the vehicle row, checks availability, calculates rental days and charge, inserts the rental, updates vehicle status, and commits the transaction. The manual specifically suggests registering a new rental and calculating the rental charge from the rental period and daily rate

```
CREATE PROCEDURE RegisterNewRental(  
    IN p_RentalID VARCHAR(20),  
    IN p_CustomerID INT,  
    IN p_VehicleNumber VARCHAR(20),  
    IN p_RentalDate DATE,  
    IN p_ReturnDate DATE  
)  
BEGIN  
    INSERT INTO Rentals (RentalID, CustomerID, VehicleNumber, RentalDate, ReturnDate)  
    VALUES (p_RentalID, p_CustomerID, p_VehicleNumber, p_RentalDate, p_ReturnDate);  
END;
```

Testing Stored Procedure Command: CALL RegisterNewRental('R005', 2, 'LMN-456', '2026-09-20', '2026-09-22');



8. Task 7 — Optimization Analysis

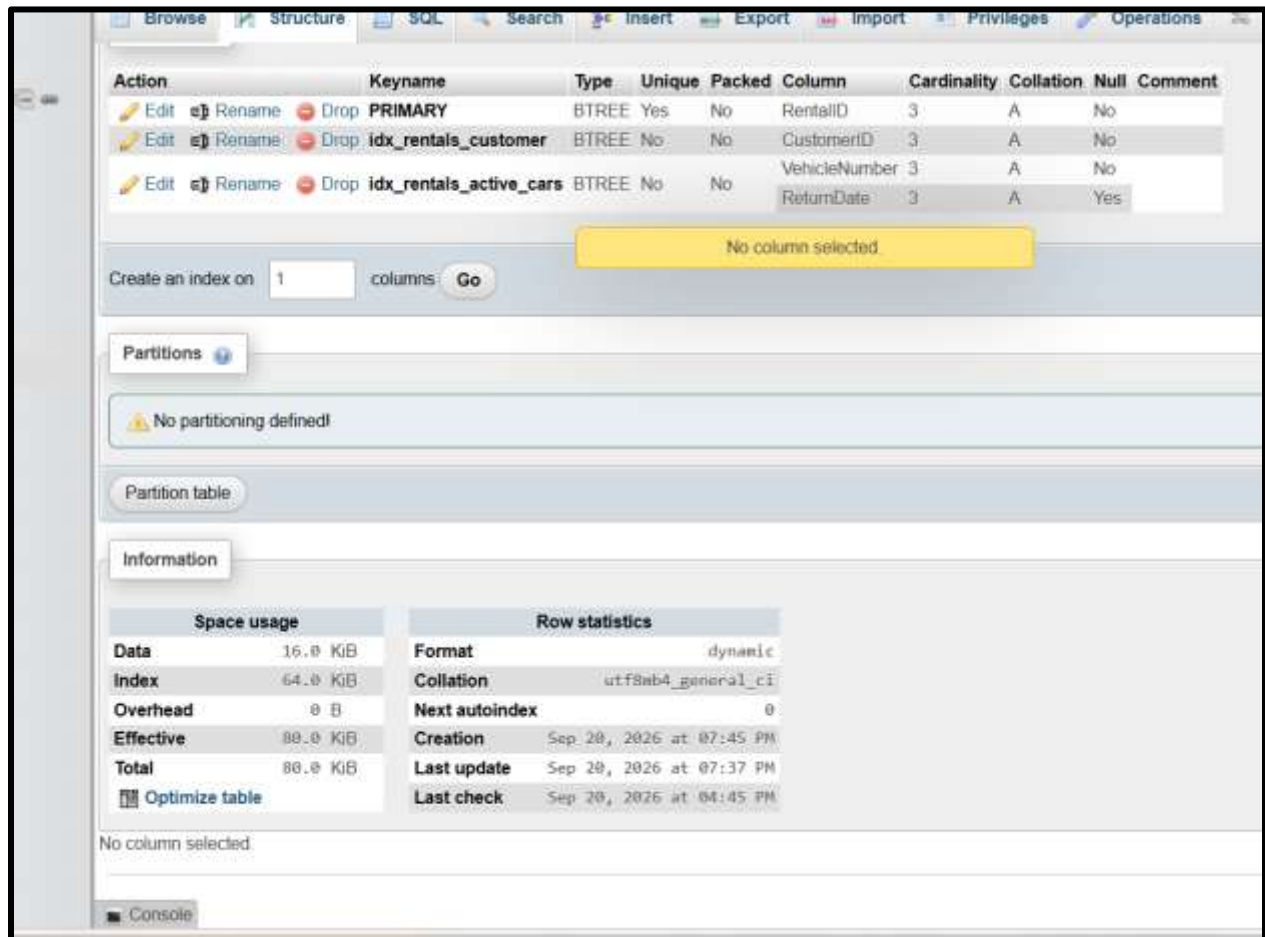
Potential inefficiency: as the Rentals table grows, JOINS and current-rental searches can become slower if the foreign-key and filtering columns are not indexed. The improved design adds indexes on CustomerID, VehicleID, (VehicleID, ReturnDate), and (CustomerID, RentalDate). The composite VehicleID/ReturnDate index is particularly relevant to the current-rental query.

The manual requires the student to identify an inefficiency, explain its performance or consistency impact, propose an improvement, and show the improved design/query.

```
CREATE INDEX idx_rentals_customer ON Rentals(CustomerID);  
CREATE INDEX idx_rentals_active_cars ON Rentals(VehicleNumber, ReturnDate);
```



Car_Rental_Management_System



9. Testing and Screenshot Checklist

The manual requires screenshots/output demonstrating JOIN queries, VIEW, TRIGGER, and STORED PROCEDURE execution. The following should be captured after running the SQL source file.

Screenshot	What to show	Expected evidence
1	Q1 output	Customer + vehicle + rental dates
2	Q2 output	All customers, including zero-rental customer
3	Q3 output	All vehicles, including currently available vehicles
4	Q4 output	Rental count for every customer
5	VIEW output	RentalReport rows with payment details
6	TRIGGER test	Attempt/return behavior and updated vehicle status

Car_Rental_Management_System

7	STORED PROCEDURE	CALL RegisterRental and resulting rental/vehicle state
---	------------------	--

10. Marking Rubric Alignment

The manual assesses five criteria, each worth up to 4 marks: Database Design & Implementation; Normalization (1NF–3NF); JOIN Queries; View, Trigger & Stored Procedure; and Optimization Analysis & Overall Quality.

Criterion	Maximum	How this submission addresses it
Database Design & Implementation	4	Logical tables, keys, constraints, realistic sample data, and justified structure.
Normalization (1NF–3NF)	4	Explicit 1NF, 2NF, and 3NF analysis with redundancy/dependency/anomaly discussion.
JOIN Queries	4	All four required queries use appropriate JOIN types.
View, Trigger & Stored Procedure	4	RentalReport VIEW, availability triggers, and transactional RegisterRental procedure.
Optimization & Overall Quality	4	Indexes, efficient current-rental JOIN, transaction/locking, and design justification.

11. Submission Files

The complete submission package contains a report and a separate SQL source file. The SQL file includes database creation, table definitions, constraints, sample data, JOIN queries, VIEW, TRIGGER, STORED PROCEDURE, testing statements, and indexes.

SQL source: Car_Rental_Management_System_Efficient.sql

Report: Car_Rental_Management_System_Complete_Report.docx

12. Conclusion

The final database provides a structured solution to the CarGo Rentals scenario while directly covering the points listed in the supplied manual. The design separates business entities, enforces integrity with keys and constraints,

Car_Rental_Management_System

reduces redundancy through normalization, answers the four required business questions, provides a reusable report view, enforces vehicle availability through triggers, automates rental registration through a stored procedure, and improves expected query performance through targeted indexes.

Important: The manual does not provide an exact schema or a fixed SQL implementation. It states that students are expected to justify their design decisions, so the SQL implementation here is a concrete design based on the requirements stated in the manual.

*****THE END*****